

抽象 · 取舍 · 定位 · 开放问题

在自主性、 可复现性与隔离性 之间做取舍

一个多智能体编排平台的设计论证。本册不讲怎么实现，只回答：这个问题域的本质困难是什么、我用了哪些抽象、这些抽象在已有工作谱系中处于什么位置、哪些问题我没有解决。

本质困难

4 个

核心抽象

8 条

对比对象

18 个

开放问题

5 个

SCOPE · 这一册讲什么

不是「我做了什么」， 是「为什么只能这么做」

一个系统的实现细节说明作者会写代码；一个系统的**约束结构**说明作者理解问题。学术面试考的是后者。本册的每一节都以「这里存在一个无法同时满足的要求」开场，以「我选了哪一边、代价是什么、什么条件下这个选择会失效」结束。

36 页

问题域的本质困难 → 核心抽象与不变式 → 可迁移的设计原则 → 在已有工作中的定位 → 评估方法与五个开放问题。

面试时以这一册为主线。

另一册 · 实现细节参考

53 页

模块、数据表、接口、协议信封、队列策略、具体文件路径。

只在被追问「那你具体怎么做的」时翻。**不要主动讲**——在学术场合先讲实现，等于宣布自己没有抽象层。

■ 三条阅读约定

- **凡是抽象，都给反例。**只说「我做了 X」没有信息量，必须说「不做 X 会怎样」。
- **凡是数字，都给来源。**没有实测过的性能数字一律不写；相关工作的数字标注论文与测量条件。
- **凡是没解决的，写进第五部分。**把局限说成未来工作是包装，说成研究问题才是能力。

一句话主张

LLM Agent 系统的工程难点不在「让模型能调工具」，而在**自主性、可复现性、可隔离性三者互相拉扯，且无法由单一计算模型同时满足**。本系统的全部结构都是这个约束的产物。

诚实前置

本系统是**工程系统，不是研究原型**：没有对照实验、没有 benchmark、没有量化评估。第五部分逐条列出缺什么、以及如果要做该怎么设计实验。

CONTENTS · 目录

五个部分

I	问题域		IV	在已有工作中的位置	
I.1	四个本质困难	06	IV.1	编排框架谱系	--
I.2	三组无法同时满足的要求	07	IV.2	持久化工作流引擎的对照	--
I.3	设计空间坐标系	08	IV.3	隔离机制谱系	--
I.4	本系统的立场	09	IV.4	协议层：MCP / ACP / A2A	--
II	核心抽象		IV.5	记忆与检索	--
II.1	二元计算模型	11	IV.6	三条定位差异	--
II.2	执行状态与调度的可重建性	12	V	评估与开放问题	
II.3	可复现性的三个层级	13	V.1	这类系统该怎么评估	--
II.4	循环的有界展开语义	14	V.2	我没有做的评估	--
II.5	类型兼容性作为协商而非拒绝	15	V.3	开放问题 RQ1-RQ3	--
II.6	能力单调收窄不变式	16	V.4	开放问题 RQ4-RQ5	--
II.7	信任域与单向断言	17	V.5	学术向问答	--
II.8	记忆表示的选择判据	18	V.6	三分钟陈述脚本	--
III	可迁移的设计原则				
III.1	可复现性需求决定配置策略	--			
III.2	fail-closed 的适用边界	--			
III.3	人在回路的粒度选择	--			
III.4	隔离机制的选择函数	--			
III.5	运行时状态必须外置	--			

NAVIGATION

如果只有十分钟：读第 4 页（主张）、II.5-II.7（三条不变式）、V.3-V.4（开放问题）。这三块决定面试官对「有没有科研潜力」的判断。

整套系统压缩成一页

CLAIM

LLM Agent 系统的工程难点，不是让模型能调用工具，而是**自主性、可复现性、可隔离性三个要求互相拉扯**；它们无法由单一计算模型同时满足，因此必须**分层**：用确定性图承载可复现与可审计，用无界循环承载自主推理，用硬件级边界承载隔离，再用**一组跨层不变式**把三者缝合。

LAYER 1

确定性图

不可变发布快照 + 持久化步骤状态。
调度是从持久状态重建的纯函数，
因此进程崩溃不丢调度进度。承载：
可审计、可续跑、可视化。

LAYER 2

无界循环

Agent 回合本身不可预测，只能用**预算**（轮次上限、超时）和**权限门**（工具级审批）截断。对上层暴露为可挂起、可快照、可取消的黑箱。

LAYER 3

硬件边界

模型生成的代码在独立内核的 microVM 内执行，第三方插件在 WASM 内执行。业务进程不持有任何宿主级权限。

■ 缝合三层的三条不变式

INV-1 · § II.6

能力单调收窄

调用链上任意子体的能力集合必须是父体的子集，任何位置都不得放宽。
把「权限正确」从「每处都写对」降为「检查每条边」。

INV-2 · § II.7

不可信方不自证

跨信任域的每个断言都由更高信任域验证：guest 的网络身份由宿主验证，回调身份由管理器验证，插件完整性由规范化内容摘要验证。

INV-3 · § II.2

状态外置于运行时

调度进度、会话快照、审批状态一律落在运行时之外的持久存储。运行时实例是可丢弃的。

可以被追问的最短版本：三层分离让每一层只解决一类问题，三条不变式保证跨层组合不会引入新的失效模式。系统的复杂度主要花在**维持这三条不变式**上，而不是功能数量上。

PART I



问题域

先说清楚这个问题为什么难。如果 Agent 编排只是「把 LLM 调用串起来」，那它不值得做成一个系统，更不值得作为研究对象。它难在四个地方，并且这四个困难两两之间还会互相加强。

I . 1

四个本质困难

I . 2

三组张力

I . 3

设计空间

I . 4

本系统立场

I.1 · INTRINSIC DIFFICULTIES

为什么它不是一个普通的分布式系统

传统工作流引擎（Airflow、Temporal）已经把「分布式任务编排」这件事解决得很好。Agent 系统之所以不能直接套用，是因为它破坏了那些引擎赖以成立的四个前提。

D1 执行是非确定性的，但恢复语义要求确定性

Temporal 这类引擎的可靠性建立在**确定性重放**之上：把历史事件重新喂给同一段代码，必须走出同一条控制流。LLM 调用天然违反这一点——同样的提示词，两次执行可能调用不同工具、循环不同轮数。于是「重放」这个最基本的恢复原语失效了，必须重新定义什么叫「恢复到同一个状态」。

D2 控制流由内容决定，图的形状运行时才知道

静态 DAG 假设「边在提交时已知」。而 Agent 的下一步取决于上一步生成的文本。要么放弃静态结构（退化为自由循环，失去可视化、可审计、可静态分析），要么强行静态化（失去自主性）。这是本领域最核心的表达力冲突。

D3 能力与风险同源，且主体意图不稳定

Agent 的价值随能力单调上升：能读文件 < 能写文件 < 能执行命令 < 能修改自身配置。风险同步上升。经典 RBAC 隐含假设「主体的意图是稳定的、可归因的」，而模型的意图既不稳定也不可归因——它可能被提示词注入改变目标。**权限模型因此不能只绑定主体，必须绑定「这一次具体操作」。**

D4 状态是多源的，「当前状态」没有单一定义

一次 Agent 执行的状态同时分布在：模型上下文窗口、工具产生的外部副作用（文件、HTTP 请求）、沙箱文件系统、待人工审批的挂起点。任何一处遗漏，「断点续跑」就只是幻觉。定义清楚**哪些状态必须持久化、哪些可以重建、哪些根本不可恢复**，是这个系统最基础的工作。

COMPOUNDING

四个困难会互相加强：D2 让图不可预测 → D4 的状态边界随之不可预测 → D1 的恢复语义更难定义 → 为了安全只能在更多地方插入 D3 的权限门 → 人工中断频率上升，自主性收益被抵消。**这个正反馈回路是整个设计的主要敌人。**

I.2 · TENSIONS

三组无法同时满足的要求

■ T1 · 自主性 ↔ 可控性

全自主

模型自行决定一切。失败模式：无限循环烧钱、不可解释的副作用、出事后无法归因。



全人工确认

每步都要点同意。失败模式：中断代价超过自动化收益，用户直接关掉这个功能。

落点：按操作的不可逆性分档，而不是按重要性。只读工具自动放行；可逆写操作记录审计；不可逆或改变系统自身配置的操作需要人工确认，且确认粒度是单次工具调用而非整个回合（§ III.3）。

■ T2 · 确定性编排 ↔ 涌现推理

纯 DAG

可视化、可审计、可续跑。失败模式：表达不了「视情况决定下一步」，退化成穿着 workflow 外衣的脚本。



纯 Agent 循环

能处理开放任务。失败模式：无结构可言，无法被非技术用户理解、无法逐步重放、无法局部重试。

落点：不做统一模型，做二元模型——外层确定性图，内层无界循环，两者通过「可挂起、可快照、可取消的会话」接口耦合（§ II.1）。代价是概念数量翻倍，用户要理解两套心智模型。

■ T3 · 能力开放 ↔ 隔离强度

进程内执行

零开销、低延迟、易调试。失败模式：模型生成的代码与业务服务同权限，一次提示词注入即宿主沦陷。



强隔离

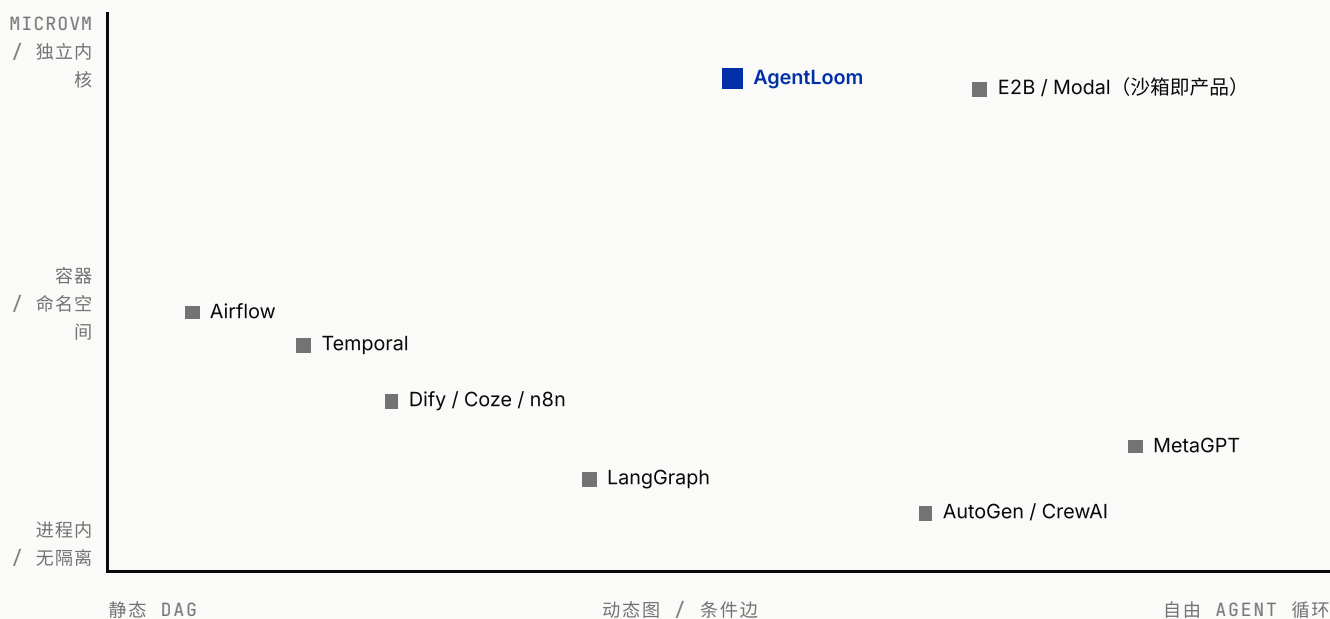
独立内核、独立网络栈。失败模式：启动开销、运维复杂度、需要特权组件，本身成为新的攻击面与单点。

落点：不选一档，而是按代码来源分档：平台自有逻辑进程内跑；第三方插件进 WASM（无系统调用面）；模型生成的代码进 microVM（独立内核）。特权收敛到单一组件（§ III.4）。

I.3 · DESIGN SPACE

两个轴就能把同类系统摆清楚

横轴是**控制流的确定程度**（编排层允许多大的运行时不确定性），纵轴是**执行隔离的强度**（对不可信代码的防护层级）。绝大多数同类系统只在其中一个轴上做深，很少有系统同时处理两轴。



READING

左下角是成熟的确定性工作流引擎——可靠但表达不了自主推理；
右下角是 Agent 框架——表达力强但执行隔离基本靠调用方自觉；
右上角是把沙箱本身做成产品的基础设施，不负责编排。

POSITIONING

本系统落在**中上区域**：编排层保持结构化（可审计、可续跑），但允许节点内部是无界循环；同时自己承担隔离责任而不外包。**这个位置的代价是系统复杂度显著高于单轴系统**，收益是能同时给出「可审计」与「能自主写代码」两个属性。

I.4 · POSITION

四条可以被证伪的立场

每条都写成可反驳的形式——如果反驳成立，对应的设计就应该推翻。

P1 Agent 与 Workflow 应当是并列实体，而不是一方包含另一方

理由：二者的生命周期、恢复边界、复用粒度都不同。工作流执行是一次性的、有确定终点、要求可审计重放；Agent 是长期存在、跨会话继续、需要独立版本与权限策略的实体。

如何证伪：如果能给出一个统一的计算模型，同时提供「逐步可重放」和「无界自主循环」，且概念复杂度不高于两套模型之和，那么这条立场就站不住。

P2 可复现性应当分级，而不是当成布尔属性

理由：LLM 输出不可复现，但这并不意味着整个系统不可复现。图结构、调度顺序、审批决策、工具调用序列都是可以复现的。把它们分开谈才能给出有意义的保证 (§ II.3)。

如何证伪：如果用户实际只关心「最终输出是否一致」，那么中间层级的可复现性就是无用的工程负担。

P3 隔离级别应当由代码来源决定，而不是由性能预算决定

理由：威胁模型不同：平台自有代码经过审查，第三方插件经过签名但逻辑不可信，模型生成的代码完全不可信且每次都不同。用同一档隔离服务三类代码，要么过度付费，要么留下缺口。

如何证伪：如果 WASM 的隔离强度对模型生成代码也够用（或 microVM 的开销可忽略），分档就没有必要。

P4 安全性应当表述为少数几条可检查的不变式，而不是一份规则清单

理由：规则清单的正确性等于「每处都记得遵守」，随功能增长必然衰减；不变式可以逐边检查，甚至可能静态验证 (§ V.3 / RQ5)。

如何证伪：如果实际系统中存在大量无法归入这几条不变式的安全逻辑，说明抽象没有覆盖真实约束，只是事后叙事。这一条目前只做到了「大部分安全逻辑可归入三条不变式」，没有做形式化验证。

PART II



核心抽象

八条抽象。判断标准是：去掉它，系统会在哪里塌。凡是去掉之后只是「代码难看一点」的，都不算抽象，只算约定。

II.1-II.2

计算模型与状态

II.3-II.4

复现性与循环

II.5

类型协商

II.6-II.8

不变式与记忆

II.1 · DUAL COMPUTATION MODEL

外层有界，内层无界

这是全系统的第一性结构：**不试图用一个模型同时表达确定性编排与自主推理，而是承认它们是两类计算，并只定义一个耦合接口。**

OUTER · 有界

确定性图

节点集有限、边在发布时冻结、拓扑序可计算、每个节点有明确的开始与结束。可以回答「跑到哪了」「还剩几步」「重跑哪一段」。

终止性：由图的有限性保证（循环另有预算上限，见 II.4）。

INNER · 无界

轮次不可预知、工具调用序列由内容决定、可能永不自然停止。**不可能从结构上保证终止**，只能从外部施加预算与权限门。

终止性：由预算（最大轮次、超时、令牌上限）与人工中断强制。

耦合接口只有四个操作

```
// 外层看到的内层，是一个黑箱状态机
create(config) → Session // 由不可变快照编译而来
load(snapshot) → Session // 崩溃后从持久状态重建
prompt(session, input) → Stream<Event>
cancel(session) → void
```

这四个操作定义了内外层的**唯一契约**。它带来两个直接后果：

- **运行时可替换**。同一接口下有两个实现——进程内运行时与 microVM 内运行时。编排层完全不知道自己调的是哪一个，因此隔离级别成了部署选择而不是代码分叉。
- **Agent 可以脱离编排单独存在**。因为接口不假设调用者是图调度器，同一个 Agent 也能被对话入口、子 Agent 递归调用复用。

COST OF THIS CHOICE

二元模型的代价是**概念不统一**：用户必须理解两套心智模型，文档要解释两遍，两层的错误处理和事件流也要各写一套。如果面试官问「为什么不像 LangGraph 那样把 Agent 也画成带环的状态图」——那是把两层压成一层的做法，好处是概念统一，代价是外层丧失「有限步内必然终止」这个性质，从而无法给出进度、剩余步数与确定的重跑边界。**这是表达力与可分析性的经典交换。**

II.2 · STATE & SCHEDULING

调度必须是持久状态的纯函数

把一次执行的状态定义为三元组：

$$E = (G, \sigma, \kappa)$$

G = 发布快照，不可变。执行期间不随编辑变化

σ : $\text{NodeId} \rightarrow \text{StepState}$ (持久化)

κ : $\text{NodeId} \rightarrow \text{Checkpoint}$ (持久化)

$\text{ready}(E) = \{ n \mid \forall p \in \text{pred}(n), \sigma(p) \in \{\text{done}, \text{skipped}\} \}$

$\text{next}(E) \subseteq \text{ready}(E)$

三个分量全部在运行时之外，于是得到一条性质：

调度状态可从持久存储完全重建。任何一个工作进程在任意时刻崩溃，新进程读库即可继续，不需要恢复内存中的任何数据结构。

■ 这条性质换来了什么

- **无需内存计数器。**「还差几个前驱完成」不缓存在内存里，每次完成事件都重新查询前驱状态。多进程并发推进同一张图时不会因计数器不同步而卡死或重复调度。
- **水平扩展是免费的。**工作进程无状态，扩副本不需要额外的一致性协议。
- **调度逻辑可单测。**`next()` 是纯函数，测试不需要起数据库和队列。

■ 代价

每个节点完成都要多做几次数据库查询。这是**用吞吐换正确性**——在 LLM 调用动辄数秒的场景里，几毫秒的查询开销可以忽略；如果节点是微秒级计算，这个设计就是错的。

BOUNDARY

「可重建」只覆盖调度层，不覆盖副作用层。若某节点已发出不可逆的外部请求后崩溃，重跑会再发一次。要做到端到端 `exactly-once`，需要节点级幂等键——本系统只有部分节点具备 (§ V.2)。

TRANSACTIONAL BOUNDARY

与之配套的一条工程约束：**任务入队必须发生在事务提交之后**。否则工作进程可能先于数据可见性读到不存在的记录。这是「消息先于数据可见」的经典陷阱，正确解法是事务性发件箱；本系统只做到提交后回调，仍存在「提交成功但入队前崩溃」的窗口。

II.3 · LEVELS OF REPRODUCIBILITY

把布尔属性拆成三级

「LLM 系统不可复现」这句话正确但没用。有用的问法是：**什么东西可复现，到什么程度，用什么代价换来的**。下面这个分级是本系统最主要的概念贡献之一。

R0 不可复现 · 无保证

两次执行之间没有任何量被保证一致。典型场景：纯 Agent 自由对话。能给出的只有事后审计——记录发生了什么，但不承诺再来一次还会这样。

R1 结构可复现 · 本系统达到的级别

保证在**相同输入与相同发布快照**下：参与执行的节点集合一致、依赖顺序一致、条件分支的判定输入可追溯、每个节点的输入输出与工具调用序列被完整记录且可逐步回看。**不保证**节点内部 LLM 的输出文本一致。

代价：图必须冻结成不可变快照（因此需要发布/草稿分离与版本表）；每步的输入必须持久化（存储成本）；事件流必须带单调序号（协议复杂度）。

R2 完全可复现 · 未实现

连模型输出都一致。需要以下之一：确定性解码 + 固定后端版本（现实中供应商无法承诺）、或对所有模型调用做记录-回放（record & replay）。后者技术上可行，是明确的未来工作（§ V.3 / RQ1）。

WHY IT MATTERS

R1 已经足够支撑三件企业级需求：**事故归因**（这一步收到了什么）、**局部重试**（从某节点及其下游重跑）、**合规审计**（谁批准了哪次工具调用）。把目标定在 R1 而不是 R2，是因为 R2 的边际成本极高而边际收益主要落在研究场景。

HONEST GAP

R1 目前是**设计意图，不是被验证的性质**——没有一致性测试去证明「同一快照两次执行的结构等价」。怎么定义并度量「结构等价」本身就是一个研究问题（RQ1）。

II.4 · BOUNDED UNROLLING

DAG 里怎么放循环

DAG 的定义是无环，而迭代天然有环。三种处理方式，代表三种取舍：

方案	机制	代价
加回边 LangGraph 类	图本身允许环，用状态机语义推进	拓扑序失效；「跑到第几步」无法定义；无法给出剩余进度；静态分析困难
事件溯源 Temporal 类	保留全部历史事件，重放推导当前状态	要求确定性重放，与 LLM 冲突；历史无限增长
有界展开 本系统	循环体是带父子层级的子图，拓扑排序时排除；每轮开始前重置子步骤状态	历史轮次的中间产物不可寻址 ——一个节点只有一行记录，只保留最后一轮的具体状态

为什么选有界展开

- 顶层图保持无环，**拓扑序、进度、剩余步数三个属性全部保住**。
- 每个节点在状态表里恒定一行，存储与查询复杂度不随轮次增长。
- 轮次信息聚合在父节点上，天然形成「循环作为一个可折叠的整体」的视图。

终止性从哪来

不来自结构，而来自**显式预算**：条件循环有最大轮次硬上限（默认 100）。这不是保守工程，而是必需——终止条件由模型判断时，「条件永远为真」是一个真实且昂贵的失效模式，不能只靠提示词约束。

TRADE-OFF, STATED

有界展开牺牲的是**历史轮次的可寻址性**。用户想问「第 3 轮的第 2 个节点输出了什么」，只能去事件流里翻，不能直接寻址。

这是一个可以改进的点：代价是把状态表从「每节点一行」变成「每节点每轮一行」，存储与索引成本随轮数线性增长。**在什么条件下这笔交换划算，取决于实际循环轮数分布——而我没有这个数据。**

II.5 · TYPES AS NEGOTIATION

把类型系统 从拒绝器改成协商器

可视化编排里，「这两个端口能不能连」是一个类型判定问题。常规做法是二值：类型相同则允许，否则拒绝。实际使用中这条规则几乎立刻失效——上游产出 `{userId, name}`，下游需要 `{id, name}`，语义完全一致但结构不同。

本系统把判定结果定义成一个**四级的有序集**，而不是布尔值：

EXACT > TRANSFORM > PARTIAL >
INCOMPATIBLE

EXACT 结构一致，直连

TRANSFORM 存在确定性转换函数

PARTIAL 部分匹配 + 缺失字段 + 候选映射

INCOMPAT 拒绝 + 冲突路径

关键设计：**PARTIAL 允许连接**。缺失字段与候选映射作为「待补证据」持久化在边上，交给用户在配置面板里确认。类型系统从「守门人」变成「协商中介」。

学术定位

这本质上是 **gradual typing** 的思路：允许一部分契约在运行时才被满足，换取开发期的可用性。代价与 gradual typing 一样——把一部分错误从编译期推迟到运行期。

两阶段判定

交互层面还有一个约束：拖拽连线的回调是同步的，不能等待异步判定。于是拆成两阶段——同步的廉价规则先挡明显错误保证跟手，异步的结构化 schema 深比较给出权威结论。这与编译器的「快速语法检查 + 完整类型检查」分层同构。

WEAKEST LINK

候选映射依赖**字段名相似度启发式**。它没有 ground truth、没有评估集、没有准确率数字。这是全系统中最像「研究问题」也最缺乏验证的部分（RQ3）。

II.6 · INVARIANT 1

能力单调收窄

对调用链上的任意父子关系 ($p \rightarrow c$):

$$\text{cap}(c) \subseteq \text{cap}(p)$$

且不存在任何运行时路径可以放宽 cap 。

一条约束，四处体现。它的价值不在于「更安全」，而在于把**权限正确性**从一个全局性质降级为一个局部可检查的性质。

■ 四处体现

- **子 Agent 不能自带沙箱**。它并入父的执行环境，而不是新开一个。否则一次对话可能拉起十几个虚拟机，且父子文件系统割裂。
- **低隔离父体不能调用高隔离子体**。进程内运行时没有能力托管一个 microVM 生命周期，这类调用在发布期就被拒绝。
- **插件运行时配置只能收紧**。平台设定超时、内存、可访问主机的上限，插件可以要求更少，不能要求更多。
- **沙箱内的 guest 不能声明自己的权限**。能力集由外部注入，guest 侧的任何自述都不被采信（见 II.7）。

■ 为什么这是抽象而不是规则

如果写成规则清单——「子 Agent 不要开沙箱」「插件别放宽超时」——那么正确性等于「每个新功能的作者都记得这几条」，必然随时间衰减。

写成不变式之后，检查对象变成**调用图上的每一条边**：只要每条边都满足 $\text{cap}(c) \subseteq \text{cap}(p)$ ，全局性质自动成立。这是一个可以在代码审查中机械执行、原则上也可以静态验证的判据。

一句话答面试：我没有让系统「更安全」，我让「是否安全」变成了一个可以逐边判定的问题。

NOT VERIFIED

当前这条不变式靠人工遵守与测试覆盖，**没有静态检查器**。给定系统的能力集合与调用图，自动验证该不变式是否处处成立，是一个明确可做的工程—研究结合点（RQ5）。

II.7 · INVARIANT 2

不可信方永远不自证

对任意跨信任域的断言 a ("我是谁"、"我能做什么"、"这份数据没被改"):

$\text{verifier}(a)$ 的信任级别 $>$ $\text{claimant}(a)$ 的信任级别

■ 四个信任域

客户端 持用户凭证与本地私钥	USER
控制面 业务服务，普通权限	CONTROL
运行时管理器 唯一特权组件	PRIVILEGED
沙箱 guest 完全不可信	UNTRUSTED

■ 三个实例

- **网络身份。** guest 声称自己的 IP/MAC 毫无意义——宿主侧在虚拟网卡入口做流量过滤，同时校验源 IP、MAC 与 ARP 发送方地址，不匹配的帧直接丢弃。这是把「网络地址」变成「不可伪造的身份」。
- **回调身份。** guest 拿不到真实的回调地址与上游令牌，只拿到一个不透明中继地址；管理器按来源地址与不透明令牌双重校验后，代持真实凭证转发到白名单主机。
- **制品完整性。** 插件签名不签 ZIP 字节，而签一份**规范化内容描述**（排序后的清单 + 逐文件哈希）。因为容器格式的字节表示不唯一，签字节等于把签名绑在打包工具上。

GENERALIZATION

这三个实例表面上毫无关系——一个是网络、一个是鉴权、一个是供应链。但它们是同一条不变式的三次应用：把「相信对方说的」替换成「由更高信任级别的一方独立验证」。

能把三件事收敛成一条规则，说明抽象抓对了。反过来，如果某处安全逻辑无法归入这条规则，就应该怀疑那里存在未识别的信任假设。

COROLLARY

推论：**密钥不跨域下发**。控制面不持有签发 guest 身份的私钥，guest 也不持有控制面的客户端证书。任一侧被攻陷都无法伪装成另一侧。

II.8 · MEMORY REPRESENTATION

相似性检索 解决不了关系性问题

系统里同时存在两套「让模型记住东西」的机制，用了两种完全不同的表示。这不是历史包袱，是按**检索目标**分的。

用途	表示	检索	判据
知识库	文本分块 + 向量	向量近邻 + 重排	目标是 相似 ：找和问题语义接近的原文片段。规模万级以上，无需版本管理
Agent 记忆	带父子边的结构化图 + 版本 + 披露级别	全文检索 + 图遍历	目标是 关系 ：「项目 A → 模块 B → 约定 C」。规模百条量级，需要版本、审阅与作废

为什么记忆不用向量库

- **结构本身就是信息**。层级与依赖关系无法由向量相似度表达，而它恰恰是复用经验时最有用的部分。
- **记忆会过时，需要作废而非删除**。版本 + 审阅状态让错误结论可被标记废弃并保留追溯，这在纯向量集合里很难表达。
- **需要披露控制**。不同场景看到不同深度的记忆。
- **规模不匹配**。百条精炼结论用全文检索足够，引入向量库只增加一个外部依赖与一致性问题。

要谨慎表述

不能说「图谱记忆优于向量检索」。现有证据是**条件性**的：GraphRAG 的论文把向量 RAG 定位为「局部事实强、全局归纳弱」，并报告在其两个语料上全局性问题的全面性与多样性占优，但**直接性 (directness) 上向量方法更好**，且图抽取与社区摘要需要额外的 LLM 索引成本。目前没有跨语料、统一模型、统一成本口径的证据表明结构化图谱全面胜过向量。

FRAMING

正确说法是：**两种表示服务两类查询**。本系统按查询类型分配表示，而不是押注某一种表示更好。